

Problemas de Búsqueda

Un Problema de búsqueda consiste en:

- Un espacio de estados
- Un modelo de transición
- Un estado inicial, Conjunto de estados finales y una función de costo.

Una solución es una secuencia de acciones (un plan) el cual transforme el estado inicial al estado final.

$S = (s_1, \dots, s_n)$ $s \in S$ Espacio de estado

$A = \{a_1, \dots, a_n\}$ acciones $\text{card}(S)$ finita

$A: S \rightarrow P(A)$

donde

$A(s) \subseteq A$ acciones legales en el estado s .

$\nwarrow$ Cardinalidad

$\text{Succ}: S \times A \rightarrow S$

$\text{costo-local}: S \times A \rightarrow \mathbb{R}^+$

$\text{costo-local}(s, a)$ es el costo de aplicar la acción a en el estado s .

Un estado inicial $s_0 \in S$

Un conjunto de estados finales $S_f \subseteq S$

Problema:

Encontrar un Plan:

$[(s_0, a_0, c_0), (s_1, a_1, c_1), \dots, (s_T, a_T, c_T)]$

donde:

$$s_{i+1} = \text{Succ}(s_i, a_i)$$

$$\text{Succ}(s_T, a_T) \in S_f$$

$$c_i = c_{i-1} + \text{costo-local}(s_i, a_i)$$

tal que

c_T son mínimo

Ejemplo Farmer, Cabbage, Goat, Wolf:

$S = \{FCGW, F, C, G, W, \emptyset\}$

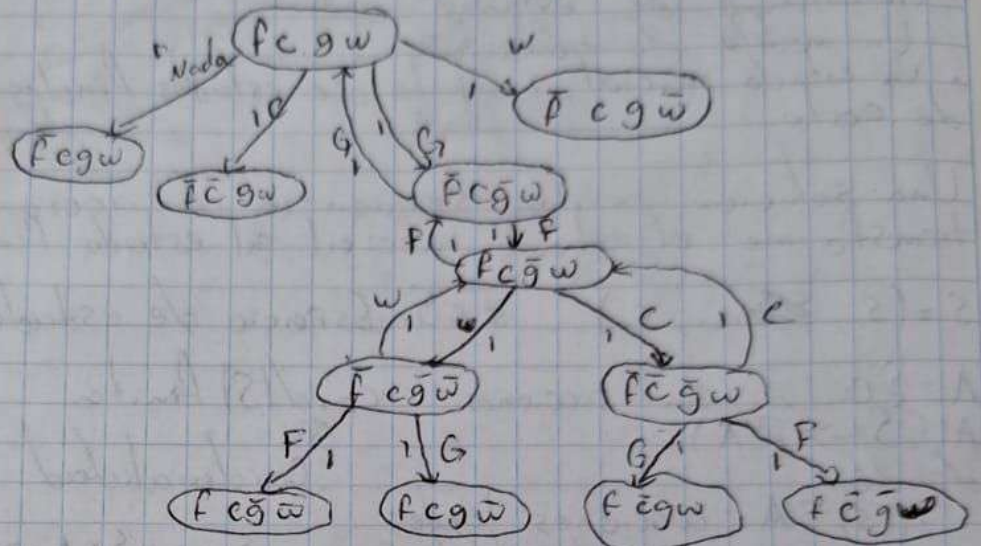
$F = \bar{F} \text{ si } \leftarrow \bar{F} \rightarrow$

$C = \bar{C} \text{ si } \leftarrow \bar{C} \rightarrow$

$G = \bar{G} \text{ si } \leftarrow \bar{G} \rightarrow$

$W = \bar{W} \text{ si } \leftarrow \bar{W} \rightarrow$

$A = \{C, G, W, \text{Nada}\}$



```
class SearchPb(Object):
    def __init__(self, s_ini):
        self.s = s_ini
    def acciones(self, s):
        raise NotImplementedError('A desarrollar')
    def sucesor(self, s, a):
        raise NotImplementedError
    def terminal(self, s):
        raise NotImplementedError
```

class abstracta



$s = (s_1, \dots, s_n)$
 $s_i \in \{A, B, C\}$

```
class TorresHanoi(SearchPb):
    def __init__(self, s0 = s * ['A'], sf = s * ['C']):
        self.s0 = s0
        self.sf = sf
        self.acciones = {'AB', 'AC', 'BA', 'BC', 'CA', 'CB'}
    def acciones(self, s):
        if any(s_i not in ['A', 'B', 'C'] for s_i in s):
            raise ValueError('Estado es imposible')
        acciones_legales = [a in self.acciones if a[0]
                             in s and (a[1] not in s or
                             s.index(a[0]) < s.index(a[1]))]
        return acciones_legales
    def sucesor(self, s, a):
        costo_local = 1
        if a not in self.acciones(s):
            raise ValueError('—')
```



```

S.next = S[0]
S.next[S.index(a[0])] = a[1]
return S.next, costo_local
def terminal(self, s):
return s == self.sf

```

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

$S = (S_0, \dots, S_5)$

$A = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$

$S_i \in \{0, \dots, 15\}$

$\text{card}(S) = \frac{16!}{2}$

```

class NPuzzle(SearchPb):
def __init__(self, n, S=None, sf=None):
    self.n = n
    if sf != None and len(sf) != n * 2:
        raise ValueError('sf y n no coinciden')
    if sf == None:
        self.S, A = [range(n * 2)]
    elif sorted(sf) != [range(n * 2)]:
        raise ValueError('sf no es una permutacion')
    else:
        self.S, A = sf[i]

```

```

def acciones(self, s):
    if not exist(self.a_legales):
        self.a_legales = []
        for i in range(self.n * 2):
            temp = []
            if i // self.n > 0:
                temp.append('u')
            if i // self.n < self.n - 1:
                temp.append('d')
            if i % self.n > 0:
                temp.append('l')
            if i % self.n < self.n - 1:
                temp.append('r')
            self.a_legales[i] = temp[i]
        return self.a_legales[S.index[0]]

```



```

def sucesor (self, s, a):
    costo_local = 1
    if a not in self.acciones(s):
        raise ValueError('___')
    s_n = s[i]
    i_vacio = s_n.index(0)
    i_otro = (i_vacio + 1 if a == 'd' else
              i_vacio - 1 if a == 'a' else
              i_vacio + self.n if a == 'd' else
              i_vacio - self.n)
    s_n[i_vacio], s_n[i_otro] = s_n[i_otro], s_n[i_vacio]
    return s_n, costo_local

```

```

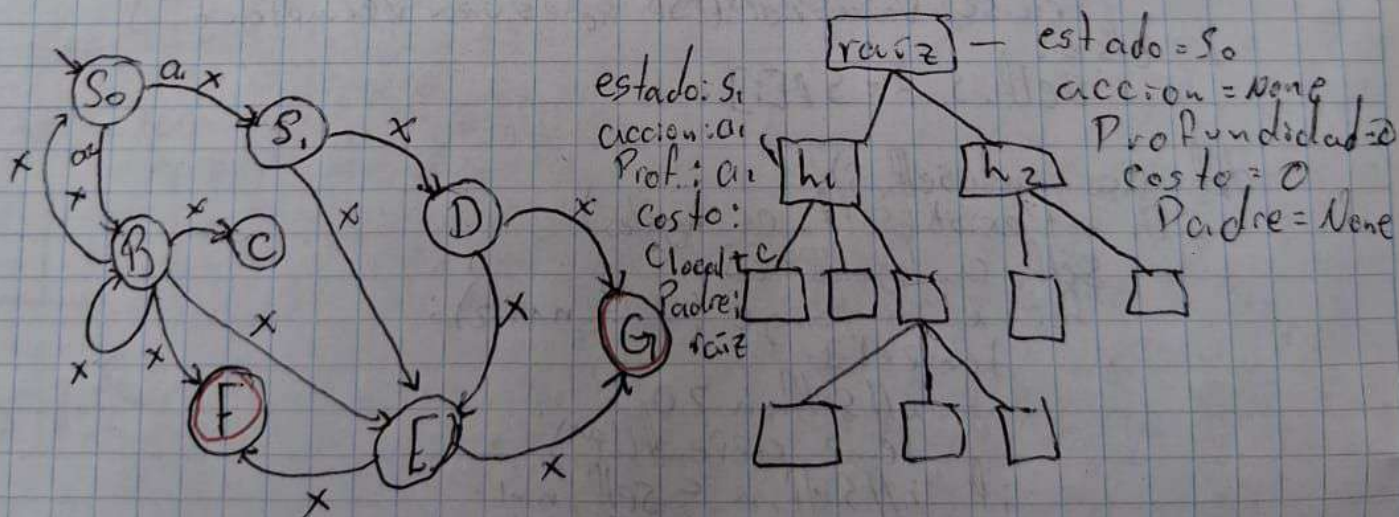
def terminal(self, s):
    return s == self.s_f

```

Plan = búsqueda (Search Pb, s₀)

Plan = [(s₀, a₀, c₀), (s₁, a₁, c₁), (s₂, a₂, c₂), ...
 (s_{T-1}, a_{T-1}, c_{T-1}), (s_T, None, None)]

donde s_T, - = sucesor (s_{T-1}, a_{T-1}) es un estado terminal



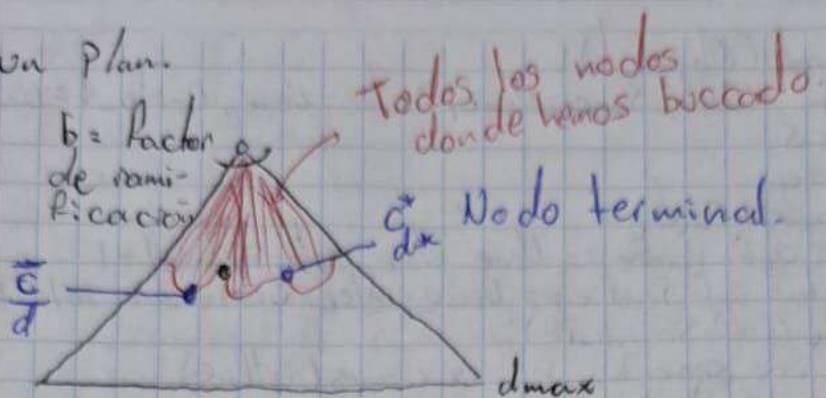
s_0
 $A(s) = \text{acciones}(s)$
 $s', \text{Costo} = \text{sucesor}(s, a)$
 $\text{terminal}(s)$


```

class Nodo Search (Object):
    def __init__(self, S, a=None, Padre=None, Costo-1=None):
        self.S = S
        self.a = a
        self.Padre = Padre
        self.d = 0, if Padre == None else self.Padre.d + 1
        self.Costo = 0, if Padre == None else Costo-1 + self.Padre.Costo
    def expande(self, Search-Pb):
        for a in Search-Pb. acciones(self.S):
            S-n, Costo-1 = Search-Pb. Sucesor(S, a)
            yield Nodo Search(S-n, a, self, Costo-1)
    def Plan(self):
        return [(self.S, None, None)] if self.Padre == None else
            self.Padre.Plan()[-1] + [(self.Padre.S, self.a, self.Costo),
            (self.S, None, None)]
    def busqueda- generica(S, Pb):
        frontera = [Nodo Search(S)]
        while frontera:
            Plan = sacar_nodo(frontera)
            if Pb. terminal(Plan.S):
                return Plan
            for Plan-hijo in Plan.expande(Pb):
                agrega-a- frontera(frontera, Plan-hijo)
        return None

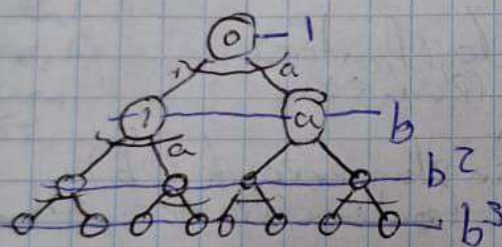
```


Tenemos un Plan.



Algoritmo:

- Admisibilidad
- Optimización
- Complejidad material
- Complejidad temporal



[0]

$$[] \rightarrow [1, \infty, a]$$
$$[2, \dots, a] \rightarrow [2, \dots, a, 11, \dots, 1a]$$
$$[3, \dots, a, 1, 1, \dots, 1, a] \rightarrow [3, \dots, a, 1, \dots, 1, a^2, \dots, a^2]$$

Si el Plan admisible de menor profundidad tiene profundidad $\geq$ entonces vamos a revisar:

Primero a lo ancho
BFS

$$1 + b + b^2 + \dots + b^d \text{ que es } O(b^d)$$

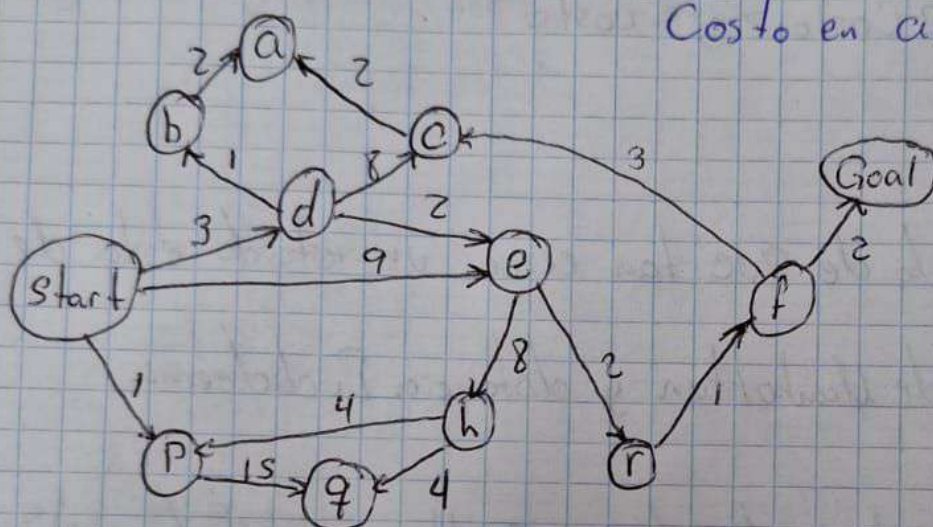
c) Admisible? - si
 d) Optimo? - solo si $e = 1$
 Temporal: $O(b^d)$
 Material: $O(b^{d-1})$

Búsqueda de grafos:

Consiste en nunca expandir un estado dos veces, así que si al estar buscando llegas a un estado que ya viste te lo saltas y revisas otro.

La búsqueda en grafos casi siempre es más rápida que la búsqueda en árboles.

Costo en acciones



BFS encuentra el camino más corto en términos del número de transiciones. NO encuentra el camino de costo más bajo.

FIRST BREADTH start $\xrightarrow{9}$ e $\xrightarrow{13}$ f $\xrightarrow{14}$ GOAL

FIRST DEPTH start $\xrightarrow{3}$ d $\xrightarrow{5}$ e $\xrightarrow{7}$ r $\xrightarrow{8}$ f $\xrightarrow{10}$ GOAL

Ejemplo de costos y movimientos

[0, START]
 [(1, p), (3, d), (9, e)]
 [(3, d), (9, e), (15, q)]
 [(4, b), (5, e), (9, e), (11, c), (15, q)]
 [(5, e), (6, a), (9, e), (11, c), (15, q)]
 [(6, a), (7, r), (9, e), (11, c), (13, h), (15, q)]
 [(7, r), (9, e), (11, c), (13, h), (15, q)]
 [(8, f), (9, e), (11, c), (13, h), (15, q)]
 [(9, e), (10, G), (11, c), (13, h), (15, q)]

g^* un plan óptimo

$\text{terminal}(g^*.estado) = \text{True}$

$g^*.costo \leq g.costo$

Para toda g tal que

$\text{terminal}(g.estado) = \text{true}$

Cuando sacamos g^* de Frontera hemos revisado **TODO**s los planes P tal que

$P.costo < g^*.costo$ y algunos que

$P.costo = g^*.costo$ para **NINGUNO**

que $P.costo > g^*.costo$

$O(b^{\lceil \frac{g^*.costo}{e} \rceil})$

Search Heuristics

• Cualquier estimado de que tan cerca un estado está de la meta.

Ejemplos: Distancia de Manhattan y distancia Euclídea

h = heurística (Plan)

h es un costo estimado de un problema relajado.

$h \leq \text{Costo de un Plan Completo desde Plan. estado}$

$h \leq g^*$ insertando en plan. estado

Si esto ocurre $\forall s \in S$ se dice que la heurística es admisible

UCS $\rightarrow (n.costo, n)$

Greedy $\rightarrow (heurística(n), n)$

$A^* (n.costo + heurística(n), n)$

$\bar{g}$ es un Plan completo que pasa por n
donde n es un Plan intermedio.

$$\bar{g}.Costo \geq n.Costo + heuristica(n)$$

$$g^* \text{ es un Plan OP + CORRECCION}^* : heuristica(\bar{g}) = 0$$

Si g^* es un Plan OPTIMO

$$heuristica(g^*) = 0$$

$$\bar{g}.Costo \leq g^*.Costo$$

$$\bar{g}.Costo \leq g^*.Costo + heuristica(g^*)$$